

PROJECT REPORT OF CO-OP Project at Industry (Module-2)

ON

Evaluating LLMs for UI Element Selection: Query Ambiguity and Interface Complexity in Developer Tools

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Harjot Singh
2210990363

Supervised By:

Dr. Gurpreet Singh
Associate Professor

Dept. of Computer Science and Engineering
Chitkara University, Punjab



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

DECLARATION

I hereby certify that the work which is being presented in the project report entitled "Evaluating LLMs for UI Element Selection: Query Ambiguity and Interface Complexity in Developer Tools" in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Gurpreet Singh. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura
Date: 26-04-2026

Harjot Singh
2210990363

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

(Dr. Gurpreet Singh)

Associate Professor
Department of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology,
Chitkara University, Punjab, India

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Dr. Gurpreet Singh, Associate Professor, Department of Computer Science and Engineering, Chitkara University, for his invaluable guidance, constant encouragement, and supervision throughout this project. His insights and suggestions were instrumental in shaping this research.

I am grateful to the faculty and staff of the Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, for providing the necessary infrastructure and resources. I also thank Google Colab for providing access to T4 GPU instances which enabled the experiments.

Finally, I extend my heartfelt thanks to my family and friends for their moral support and encouragement throughout this journey.

Harjot Singh

Roll Number: 2210990363

ABSTRACT

In recent years, Large Language Models (LLMs) have found increasing applications in helping users navigate software interfaces. However, no research has looked into the ability of LLMs in accurately detecting the proper User Interface (UI) element for different levels of query ambiguity and interface complexity in the context of developer tools. An experimental evaluation of five open-source LLMs — TinyLlama (1.1B), Gemma 2 (2B), Phi-3 Mini (3.8B), Llama 3 (8B) and Mistral (7B) — is conducted in this paper using a set of 360 structured queries derived from GitHub and AWS interfaces.

The data follows a 3x2 factorial experiment design, where three factors of ambiguity (clear, paraphrased, ambiguous) and two types of interface sizes (10 elements and 21-24 elements) were used. Among the tested models, Mistral 7B yields the best performance of 90.00%, whereas TinyLlama (1.1B) performs randomly (7.22%) and therefore forms a lower bound on the parameter number required for this task category. Overall, accuracy decreases systematically as the query ambiguity level rises, which is amplified further when dealing with large interfaces. In addition, performance between Gemma 2 (2B) and Llama 3 (8B) cannot be distinguished significantly ($p = 0.8918$), suggesting that instruction tuning might be more crucial than pure model size for structural UI selection tasks.

Keywords: Large Language Models, UI element selection, query ambiguity, interface complexity, developer tools, GitHub, AWS, LLM evaluation

CONTENTS

S. No.	Title	Page No.
	Abstract	iv
	List of Figures and Tables	v
1	Introduction	1
2	Literature Survey & Research Gap Identification	2
3	Dataset Design	4
4	Methodology	5
5	Results	6
6	Discussion	9
7	Threats to Validity	10
8	Conclusion and Future Scope	11
	References	12

LIST OF FIGURES AND TABLES

List of Tables

Table No.	Title	Page No.
Table I	Dataset Statistics	4
Table II	Models Evaluated	5
Table III	Full Performance Metrics	6
Table IV	Accuracy by Ambiguity Level (%)	7
Table V	Accuracy by Interface Size (%)	7
Table VI	McNemar's Pairwise Significance Test	8

List of Figures

Figure No.	Title	Page No.
Fig. 1	Overall accuracy of all five models with random-chance baselines	6
Fig. 2	Accuracy by query ambiguity level	7
Fig. 3	Accuracy by interface complexity	7
Fig. 4	Heatmap: accuracy at each ambiguity $\times$ interface size combination	8
Fig. 5	Accuracy by platform (GitHub vs. AWS)	8
Fig. 6	Error distribution by ambiguity level per model	9
Fig. 7	Radar chart: multi-dimensional performance profiles	9
Fig. 8	Per-task accuracy (Task IDs 1–60)	9

1. Introduction

Modern software development frameworks like GitHub and Amazon Web Services involve interfaces with a considerable amount of interactive options — sometimes tens or even hundreds of them. In order for modern Large Language Models (LLMs) used in agent-based systems, coding helpers, and autonomously acting user interfaces to be effective, it is important for them to understand what the user wants to accomplish and pick the correct interface element accordingly. However, doing so entails two major problems: the clarity of the user's request and the number of available options on the screen.

It is a known issue that a natural language processing model faces when trying to understand user requests because a person can say "create a new repository", "start a new project repo," or even just "set something up for my code" to accomplish the same thing. With multiple options available (e.g., twenty or more), the model would have to both recognize what the user wanted and pick out the one option that best accomplishes the given task from several possibilities.

Despite growing interest in the potential of LLM-powered interface agents, there is currently insufficient work that empirically analyses how the combination of query ambiguity and interface complexity impacts various model behaviours on modern developer interfaces. The current work seeks to fill this knowledge gap.

The key contributions of this study are: (1) a balanced dataset consisting of 360 query instances for 60 different tasks in GitHub and AWS interfaces arranged in a 3x2 factorial structure; (2) an experimental comparison of five open-source LLMs with parameter sizes ranging between 1.1B and 8B in a zero-shot setting; (3) statistical comparison of all 10 possible model pairs using McNemar's test; (4) empirical analysis of minimum parameter threshold and role of instruction-tuning; and (5) a research gap analysis section dedicated to evaluating current contributions within the field.

2. Literature Survey and Research Gap Identification

A. LLM Evaluation Benchmarks

The benchmarking of large language models started with GLUE and later SuperGLUE, providing a common ground to evaluate models in their ability to handle natural language understanding tasks like textual entailment, question answering, and co-reference. These benchmarks showcased the potential of transformers to generalize across a wide array of tasks via fine-tuning. The BIG-Bench suite expanded the scope of benchmarking to over 200 tasks that test for abilities emerging beyond regular NLP tasks, while MMLU tested models on university level knowledge across 57 subjects, demonstrating clear differences in performance amongst different families of models. HellaSwag and WinoGrande focused on commonsense reasoning, highlighting that even large models struggle with subtle inferences requiring world knowledge. Notwithstanding, these benchmarks only cover open-ended generation or multiple-choice reasoning problems; none addresses the closed-set item selection characteristic of UI interaction tasks.

B. LLMs for User Interface and Web Agent Tasks

The use of neural models in interpreting and interacting with user interfaces has gained tremendous popularity. Shi et al. proved that web-based task completion using neural networks is possible in the Worlds of Bits domain. Mind2Web provided an extensive dataset for training web agents through real-world task completion involving 2,000 tasks across 137 websites. WebArena and VisualWebArena offered realistic multi-application web environments for evaluating autonomous agents. SeeAct offered a solution using GPT-4V through iterative processes of visual perception and action grounding. All these approaches focus on end-to-end task completion using powerful closed-source models, whereas the current study focuses on the sub-task of element selection in a single-step process using lightweight open-source models.

C. Instruction Following in Small and Open-Source LLMs

The Mistral 7B architecture shows that with proper design, a 7B-parameter model with sliding window attention can perform as well as much larger models. The instruction-fine-tuned version achieves excellent results on MT-Bench and AlpacaEval tests. Microsoft's Phi-3 Mini (3.8B) achieves comparable MMLU scores relative to models with five times more parameters using high-quality synthetic training data. Gemma 2 (2B) by Google performs knowledge distillation

from larger teacher models, producing impressive reasoning abilities. TinyLlama (1.1B) represents the lowest threshold of deployable open-source solutions.

D. Query Ambiguity in Natural Language Processing

Query ambiguity has received considerable attention in information retrieval literature. Sanderson provided a classification of query ambiguities in web search, classifying them into ambiguity induced by polysemy, underspecification, and implicit context. In dialogue systems, Schlangen and Skantze formulated an incrementally interpretable model for ambiguity resolution. Jiang et al. showed that minor modifications in the vagueness of instructions result in steep declines in task success rates. Shi et al. investigated the effect of query paraphrasing on LLM-based information retrieval, showing that query reformulation impairs retrieval accuracy despite maintaining semantic equivalence.

E. Research Gap Identification

Based on the literature review, five distinct research gaps were identified:

- Gap 1: No controlled study of query ambiguity as an isolated variable in UI element selection tasks.
- Gap 2: Interface complexity has not been used as a controlled experimental variable in prior studies.
- Gap 3: Sub-8B open-source models are under-evaluated on structured UI interaction tasks.
- Gap 4: Pairwise statistical significance testing between models is absent from most benchmarking literature.
- Gap 5: Developer-focused platforms (GitHub, AWS) are excluded from existing UI agent evaluation datasets.

3. Dataset Design

A. Overview

The dataset consists of 360 queries based on 60 unique base tasks, evenly split into 30 tasks for GitHub and 30 tasks for AWS. Each base task is evaluated under the following experimental settings: three levels of query ambiguity (clear, paraphrased, ambiguous) and two interface sizes (small: 10 elements; large: 21-24 elements) resulting in six conditions for each base task. This design directly addresses Gap 1 and Gap 2.

Table I: Dataset Statistics

Property	GitHub / AWS	Total
Base tasks	30 / 30	60
Total rows	180 / 180	360
Ambiguity levels	3 each	120/level
Interface sizes	2 each	180/size
Validation checks	18/18 passed	Pass

B. Ambiguity Levels

Clear queries use direct, technical language matching the name of the UI element (e.g., "Create a new repository"). Paraphrased queries express the same intention by substituting synonyms or altering syntax (e.g., "Duplicate someone else's repository to my profile"). Ambiguous queries feature vague and intent-implied language which may reasonably correspond to several UI elements (e.g., "Work on someone else's project independently"). This trinary categorization relies on the taxonomy presented by Sanderson.

C. Interface Conditions

Small interfaces include precisely 10 UI elements, consisting of the right answer and 9 distractors. In large interfaces, there are 21 to 24 UI elements with 20 to 23 distractors. The probability of guessing correctly at random is 10% and 4.5% for small and large interfaces respectively. Such design directly tests for the identified interaction effect in Gap 2.

D. Validation

The dataset underwent 18 automatic validation procedures regarding its structural correctness, class balance, semantic consistency, uniqueness across tasks, and inclusion of the right answer among the options. All 18 checks passed.

4. Methodology

A. Model Selection

Five models with parameter counts in the range from 1.1B to 8B were chosen to resolve Gap 3. TinyLlama (1.1B) tests the lower bound for viability. Gemma 2 (2B) and Phi-3 Mini (3.8B) serve to test efficiency claims at the middle range. Llama 3 (8B) and Mistral 7B represent lightweight open-source models in the higher category.

Table II: Models Evaluated

Model	Params	Developer	Acc. %	F1-W %
TinyLlama	1.1B	TinyLlama	7.22	9.70
Phi-3 Mini	3.8B	Microsoft	69.72	74.32
Gemma 2	2B	Google	81.67	84.04
Llama 3 8B	8B	Meta	82.22	84.55
Mistral 7B	7B	Mistral AI	90.00	91.05

B. Prompt Design

All models were queried with a zero-shot system prompt instructing them to select just one UI element by its name, without further explanation. The user prompt included the platform name, interface size, the query, and numbered options. The temperature parameter was set to 0 for deterministic outputs. This controlled prompt design isolates the effect of query ambiguity — the variable identified as unstudied in Gap 1 — by holding all other prompt variables constant across all 360 instances.

C. Factorial Design

The 3x2 factorial structure simultaneously varies ambiguity level and interface size across the same set of 60 tasks, enabling measurement of both main effects and their interaction. This design directly resolves Gap 2 by producing clean pairwise comparisons between small and large interfaces under identical query conditions.

D. Inference Setup

Models ran via Ollama on a Google Colab T4 GPU instance (16 GB VRAM). Each model was loaded once and queried sequentially across all 360 rows. Raw outputs were cleaned via exact match, substring match, and partial heuristic before scoring.

E. Evaluation Metrics

Primary metric is accuracy (exact case-insensitive match). Secondary metrics include weighted F1, weighted precision, and weighted recall. Statistical significance is assessed via McNemar's paired test on per-row correctness vectors (threshold $p < 0.05$) for all 10 model pairs. This directly resolves Gap 4 by replacing leaderboard-style accuracy comparisons with formal hypothesis testing.

5. Results

A. Overall Accuracy

Table III shows full performance metrics. Mistral 7B leads at 90.00% accuracy and 91.05% weighted F1. Llama 3 8B (82.22%) and Gemma 2 2B (81.67%) are the next strongest. Phi-3 Mini 3.8B lags at 69.72% despite more parameters than Gemma 2. TinyLlama 1.1B at 7.22% sits within random-chance range.

Table III: Full Performance Metrics

Model	Acc %	F1-Mic	F1-Mac	F1-W	Prec-W
TinyLlama	7.22	7.22	2.26	9.70	18.42
Phi-3 Mini	69.72	69.72	44.15	74.32	84.16
Gemma 2 2B	81.67	81.67	65.49	84.04	90.63
Llama 3 8B	82.22	82.22	62.63	84.55	90.58
Mistral 7B	90.00	90.00	78.05	91.05	93.69

B. Effect of Query Ambiguity

Table IV shows consistent monotonic accuracy decline from clear to ambiguous for all capable models. Mistral 7B drops 19.17 percentage points (99.17% to 80.00%). TinyLlama shows no meaningful signal at any ambiguity level, confirming its inability to follow structured instructions.

Table IV: Accuracy by Ambiguity Level (%)

Model	Clear	Paraphrased	Ambiguous	Drop C→A
TinyLlama	14.17	5.83	1.67	-12.50
Phi-3 Mini	88.33	69.17	51.67	-36.66
Gemma 2 2B	95.83	83.33	65.83	-30.00
Llama 3 8B	96.67	86.67	63.33	-33.34
Mistral 7B	99.17	90.83	80.00	-19.17

C. Effect of Interface Complexity

Every model performs better on small interfaces than large ones. Phi-3 Mini shows the largest drop of 17.22 percentage points; Mistral 7B shows the smallest at 6.66 percentage points. The worst condition is ambiguous + large: Phi-3 Mini drops to 41.67% and Llama 3 to 53.33%.

Table V: Accuracy by Interface Size (%)

Model	Small (10)	Large (21-24)	Drop S→L
TinyLlama	10.00	4.44	-5.56
Phi-3 Mini	78.33	61.11	-17.22
Gemma 2 2B	84.44	78.89	-5.55
Llama 3 8B	88.89	75.56	-13.33
Mistral 7B	93.33	86.67	-6.66

D. Effect of Platform

All models perform moderately better on AWS than GitHub. AWS actions use consistent action-object naming (e.g., 'Create S3 Bucket') which better matches model output patterns. Platform differences are smaller than those induced by ambiguity or interface size.

E. Statistical Significance

Table VI presents all 10 McNemar's test results. Nine of ten pairs are significant at $p < 0.05$. The only non-significant pair is Gemma 2 2B vs. Llama 3 8B ($p = 0.8918$) — a 0.55 point difference that is statistically indistinguishable despite a 6B parameter gap between models. This directly resolves Gap 4.

Table VI: McNemar's Pairwise Significance Test

Model Pair	p-value	Significant?
Phi-3 Mini vs. Gemma 2 2B	< 0.0001	Yes
Phi-3 Mini vs. TinyLlama	< 0.0001	Yes
Phi-3 Mini vs. Mistral 7B	< 0.0001	Yes
Phi-3 Mini vs. Llama 3 8B	< 0.0001	Yes
Gemma 2 2B vs. TinyLlama	< 0.0001	Yes
Gemma 2 2B vs. Mistral 7B	< 0.0001	Yes

Gemma 2 2B vs. Llama 3 8B	0.8918	No
TinyLlama vs. Mistral 7B	< 0.0001	Yes
TinyLlama vs. Llama 3 8B	< 0.0001	Yes
Mistral 7B vs. Llama 3 8B	0.0001	Yes

6. Discussion

A. Instruction-Tuning Quality Over Model Size

Gemma 2 2B achieves statistical parity in terms of accuracy ($p = 0.8918$) with that of Llama 3 8B but uses just one-fourth of the number of parameters. This result supports recent studies suggesting that the quality of instruction tuning data is more important than the sheer number of parameters in tasks with strict constraints. On the other hand, Phi-3 Mini serves as an example where a large number of parameters and wide context window fail to provide good results, possibly because of its emphasis on mathematical calculations and coding.

B. Minimum Parameter Threshold (Gap 3 Resolution)

TinyLlama displays near-random behavior, scoring 7.22% against a 7% baseline. Such a trend indicates the existence of a threshold phenomenon: models with approximately 1.5–2B instruction-tuned parameters are inadequate for this task. These results directly address Gap 3 by determining the empirical lower limit for applying sub-8B models to UI agent tasks.

C. Ambiguity Is the Dominant Failure Mode (Gap 1 Resolution)

The accuracy reduction caused by ambiguity (maximum 36.6 percentage points) for all models exceeds the accuracy reduction because of interface complexity (maximum 17.22 percentage points) more than twice. This result conclusively resolves Gap 1: ambiguity is the leading cause of failure rather than a secondary contributor. In light of this, future UI agent designs must focus on addressing query disambiguation.

D. Compound Degradation Effect (Gap 2 Resolution)

Heatmaps prove that the relationship between ambiguity and interface complexity has a non-linear nature. Mistral 7B sees the accuracy drop by 23.3 percentage points when transitioning from optimal conditions (clear + small: 100%) to unfavourable ones (ambiguous + large: 76.7%), a figure greater than the sum of individual drops. This multiplicative effect would not be detected without the factorial methodology, thus conclusively resolving Gap 2.

7. Threats to Validity

A. Construct Validity

There might be a labelling bias due to the creation of the dataset by the researchers themselves. Further studies could incorporate crowd-sourced queries or mined logs.

B. Internal Validity

For all experiments, a single zero-shot prompting template is employed. The results may vary for few-shot examples or chain-of-thought prompting; hence, the conclusions can only apply to this specific prompting strategy.

C. External Validity

This experiment considers only two platforms, namely GitHub and AWS. Whether the conclusions hold true in consumer products, mobile interfaces, or non-English language interfaces requires separate evaluation.

D. Reliability

The temperature value was held constant at 0 to obtain deterministic outputs. All model weights were retrieved from the Ollama registry in their default versions and are documented for full reproducibility.

8. Conclusion and Future Scope

A. Conclusion

The current work carries out an empirical analysis of five open-source large language models in the task of user interface element selection within GitHub and AWS developer interfaces using a balanced dataset of 360 examples in a 3x2 factorial design. Five gaps are identified in the existing literature and systematically addressed: (1) lack of ambiguity control in UI selection tasks, (2) exclusion of interface complexity from experimental variables, (3) insufficient evaluation of sub-8B parameter models, (4) lack of formal pairwise statistical testing, and (5) exclusion of developer platforms from UI agent datasets.

Mistral 7B shows the best performance with accuracy equal to 90.00%. Models with less than approximately 2B parameters do not perform better than random choice. Query ambiguity proves to be the main factor leading to mistakes and, importantly, the effect is strongly non-linearly amplified by interface complexity. Gemma 2 2B shows statistical parity to Llama 3 8B with one-quarter of its parameters, indicating that the quality of instruction tuning plays a decisive role.

B. Future Scope

As future work, the study suggests the following directions for further research:

- Investigating few-shot prompting and chain-of-thought reasoning for UI element selection.
- Developing query disambiguation pre-processing pipelines for ambiguous natural language inputs.
- Fine-tuning lightweight models on developer UI vocabularies to improve structured selection accuracy.
- Expanding the benchmark to mobile platforms and non-English language interfaces.
- Including consumer-facing application platforms to test generalizability of findings.
- Exploring multi-modal approaches combining visual UI rendering with natural language processing.

References

- [1] I. Gur, H. Furuta, A. Huang, M. Safdari, Y. Matsuo, D. Eck, and A. Faust, "Real-World WebAgent with Planning, Long Context Understanding, and Program Synthesis," arXiv:2307.12856, 2024.
- [2] S. Yao, H. Chen, J. Yang, and K. Narasimhan, "WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents," arXiv:2207.01206, 2022.
- [3] A. Wang et al., "GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding," arXiv:1804.07461, 2019.
- [4] A. Wang et al., "SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems," arXiv:1905.00537, 2019.
- [5] BIG-Bench Collaboration, "Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models," arXiv:2206.04615, 2023.
- [6] D. Hendrycks et al., "Measuring Massive Multitask Language Understanding," arXiv:2009.03300, 2021.
- [7] T. Shi, A. Karpathy, L. Fan, J. Hernandez, and P. Liang, "World of Bits: An Open-Domain Platform for Web-Based Agents," PMLR 70:3135-3144, 2017.
- [8] X. Deng et al., "Mind2Web: Towards a Generalist Agent for the Web," arXiv:2306.06070, 2023.
- [9] S. Zhou et al., "WebArena: A Realistic Web Environment for Building Autonomous Agents," arXiv:2307.13854, 2023.
- [10] A. Q. Jiang et al., "Mistral 7B," arXiv:2310.06825, 2023.
- [11] M. Abdin et al., "Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone," arXiv:2404.14219, 2024.
- [12] M. Sanderson, "Ambiguous Queries," in Proceedings of ACM SIGIR, 2008.
- [13] D. Schlangen and G. Skantze, "A General, Abstract Model of Incremental Dialogue Processing," EACL, 2009.
- [14] J. Jiang et al., "Evaluating the Effect of Instruction Vagueness on Task Success in Spoken Dialogue," SIGDIAL, 2019.

- [15] R. Zellers et al., "HellaSwag: Can a Machine Really Finish Your Sentence?" arXiv:1905.07830, 2019.
- [16] K. Sakaguchi et al., "WinoGrande: An Adversarial Winograd Schema Challenge at Scale," arXiv:1907.10641, 2019.
- [17] J. Koh et al., "VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks," arXiv:2401.13649, 2024.
- [18] Y. Zheng et al., "GPT-4V(ision) is a Generalist Web Agent, if Grounded," arXiv:2401.01614, 2024.
- [19] Y. Wang et al., "Screen2Words: Automatic Mobile UI Summarization with Multimodal Learning," UIST, 2021.
- [20] Y. Li et al., "ActionBert: Leveraging User Actions for Semantic Understanding of User Interfaces," AAAI, 2021.
- [21] A. Grattafiori et al., "The Llama 3 Herd of Models," arXiv:2407.21783, 2024.
- [22] Google DeepMind, "Gemma 2: Improving Open Language Models at a Practical Size," arXiv:2408.00118, 2024.
- [23] P. Zhang et al., "TinyLlama: An Open-Source Small Language Model," arXiv:2401.02385, 2024.
- [24] C. Carpineto and G. Romano, "A Survey of Automatic Query Expansion in Information Retrieval," ACM CSUR, 2012.
- [25] F. Shi et al., "Detecting Hallucinations in Large Language Model Generation," arXiv:2303.08896, 2023.